

Spring Boot Fundamentals - Class 1 Notes

What is Spring Boot?

Spring Boot is a framework that simplifies the development of Spring-based applications. It provides:

- **Auto-configuration:** Automatically configures your application based on dependencies
- **Starter dependencies:** Pre-configured dependency sets for common use cases
- **Embedded servers:** Built-in Tomcat, Jetty, or Undertow servers
- **Production-ready features:** Metrics, health checks, and externalized configuration

What is a Framework?

A framework is a collection of libraries and tools that provide a foundation for building applications. It:

- Provides a structure for your application
- Handles common tasks automatically
- Enforces best practices and patterns
- Reduces boilerplate code

Example: Spring Boot is a framework that handles web server setup, dependency injection, and configuration management.

What is a Library?

A library is a collection of pre-written code that you can use in your application. It:

- Provides specific functionality
- Requires you to integrate it manually
- Gives you more control but requires more setup
- Examples: Jackson (JSON processing), Hibernate (ORM)

Framework vs Library:

- Framework calls your code
- You call library code

History of Spring Boot

Evolution Timeline:

1. **2003:** Spring Framework 1.0 released
 - Dependency Injection and IoC container
 - Required extensive XML configuration
2. **2004-2010:** Spring Framework evolution
 - Added annotations support
 - Reduced XML configuration
 - Still complex setup
3. **2014:** Spring Boot 1.0 released
 - Created by Pivotal (now VMware)
 - "Convention over configuration" approach
 - Auto-configuration and starters
4. **2015-2020:** Rapid adoption
 - Became the de facto standard for Java web applications
 - Microservices architecture support
5. **2021-Present:** Modern Spring Boot
 - Native compilation support
 - Cloud-native features
 - Reactive programming support

Using Spring Initializr (start.spring.io)

Spring Initializr is a web-based tool that generates Spring Boot project templates:

Steps:

1. Go to <https://start.spring.io>
2. Select project metadata:
 - **Project:** Maven or Gradle

- **Language:** Java
- **Spring Boot version:** Latest stable
- **Group:** com.example (or your organization)
- **Artifact:** your-project-name
- **Name:** Your project name
- **Description:** Brief description
- **Package name:** com.example.yourproject
- **Packaging:** Jar
- **Java version:** 17 or 21

3. Add dependencies:

- **Spring Web:** For REST APIs
- **Spring Data JPA:** For database operations
- **H2 Database:** For development
- **Spring Boot DevTools:** For development features

4. Click "Generate" to download the project

IntelliJ IDEA Setup

Installation:

1. Download IntelliJ IDEA Community Edition (free)
2. Install with default settings
3. Install Spring Boot plugin (if not included)

Project Setup:

1. Open IntelliJ IDEA
2. Select "Open" and choose your downloaded project folder
3. Wait for Maven/Gradle to download dependencies
4. IntelliJ will automatically detect Spring Boot project

Useful IntelliJ Features:

- **Code completion:** Ctrl+Space
- **Run application:** Right-click on main class → Run
- **Debug:** Set breakpoints and debug
- **Live templates:** Type "sout" → Tab for System.out.println()

Basic Project Structure

```
your-project/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── example/
│   │   │   │   │   ├── yourproject/
│   │   │   │   │   │   ├── YourProjectApplication.java
│   │   │   │   │   │   └── resources/
│   │   │   │   │   │       ├── application.properties
│   │   │   │   │   │       ├── static/
│   │   │   │   │   │       └── templates/
│   │   └── test/
│   │       ├── java/
│   │       │   ├── com/
│   │       │   │   ├── example/
│   │       │   │   │   ├── yourproject/
│   │       │   │   │   │   ├── YourProjectApplicationTests.java
│   │       └── target/ (Maven build output)
│   └── pom.xml (Maven configuration)
└── README.md
```

Key Files:

- **YourProjectApplication.java:** Main class with @SpringBootApplication
- **application.properties:** Configuration file
- **pom.xml:** Maven dependencies and build configuration

Running Your First Application

Method 1: IntelliJ IDEA

1. Right-click on main class
2. Select "Run 'YourProjectApplication'"
3. Check console for "Started YourProjectApplication"

Method 2: Command Line

```
# Navigate to project directory
cd your-project

# Run with Maven
./mvnw spring-boot:run

# Or build and run JAR
./mvnw clean package
java -jar target/your-project-0.0.1-SNAPSHOT.jar
```

What Happens When You Run:

1. Spring Boot starts embedded Tomcat server
2. Default port: 8080
3. Application context loads
4. Auto-configuration kicks in
5. Application is ready to accept requests

Verify It's Working:

- Open browser: <http://localhost:8080>
- Check console logs for startup messages
- Look for "Tomcat started on port(s): 8080"

Next Steps

In the next class, we'll:

- Create our first REST controller
- Understand Spring Boot's auto-configuration
- Learn about application properties
- Build a simple "Hello World" API

Key Takeaways:

- Spring Boot simplifies Spring development
- Frameworks provide structure, libraries provide functionality
- Spring Initializr generates project templates
- IntelliJ IDEA is the recommended IDE
- Basic project structure follows Maven conventions
- Running applications is straightforward with embedded servers